



Modularità

Organizzare il codice sorgente

Lezione 19-
Corso 1



Crates

- Il processo di compilazione di un progetto Rust è basato sul concetto di **crate**
 - A livello sorgente, un crate è un'unità di compilazione (insieme di file che costituiscono il progetto)
 - La compilazione di ciascun file appartenente al crate dà origine ad una file oggetto che potrà essere collegato (linked) con altri file oggetto per diventare un **eseguibile** completo (dotato di un punto di ingresso esplicito, `main()` , e di tutte le sue dipendenze statiche) oppure diventare una **libreria**
- Un crate binario costituisce un'unità di versionamento e di caricamento in fase di esecuzione
 - Quando un crate fa riferimento a codice (funzioni, strutture, tratti, ...) esportato da un altro crate (di tipo libreria), quest'ultimo deve essere combinato con il primo per mettere a disposizione quanto richiesto
 - Questa operazione è detta collegamento e dipende da come la libreria è stata realizzata
- Rust supporta la creazione di librerie con diverse forme di collegamento
 - Statico, in cui l'eseguibile finale ha una copia completa di tutto il codice utilizzato
 - Dinamico, in cui il codice esterno viene caricato all'atto dell'avvio dell'eseguibile, a partire da un file di tipo `.dll` (in Windows) o `.so` (nei sistemi Unix-like)

Librerie Statiche vs. Librerie Dinamiche

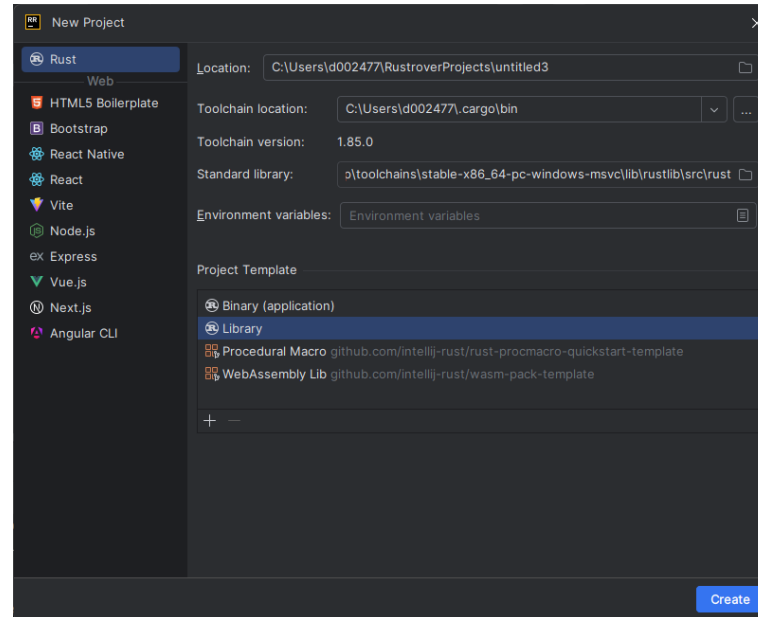
- Libreria Statica: il codice della libreria statica viene copiato direttamente all'interno del file eseguibile del programma durante la fase di compilazione e linking. Durante l'esecuzione tutto il codice necessario è già presente all'interno dell'eseguibile.
- Libreria dinamica: durante la compilazione e il linking nel file eseguibile viene inserito solo un riferimento alla libreria dinamica e il codice della libreria non viene copiato al suo interno. La libreria dinamica viene caricata in memoria solo quando il programma viene eseguito. Il Sistema Operativo si preoccupa di trovare e caricare la libreria. Più processi possono condividere la stessa copia della libreria dinamica in memoria, portando ad un uso più efficiente delle risorse. Le librerie dinamiche sono mappate dal Sistema Operativo in uno spazio degli indirizzi di memoria riservate al Sistema Operativo stesso.

Librerie

- La sezione **[lib]** del file Cargo.toml che descrive il progetto contiene dettagli relativi al tipo di libreria che si intende creare. Occorre specificare l'opportuno valore della chiave **crate-type** per ottenere i diversi seguenti tipi di librerie:
 - **rlib**: libreria statica Rust (crate-type = ["lib"], **valore di default**) il file risultante può solo essere usato da altri eseguibili Rust che ne includeranno le parti importate direttamente nel proprio codice binario
 - **dylib**: libreria dinamica Rust (crate-type = ["dylib"]); viene trasformata in un file .dll in windows, .so in linux, .dylib in macOS; può essere utilizzata solo da codice Rust, ma sarà caricata in fase di esecuzione
 - **staticlib**: libreria statica conforme alle specifiche C (crate-type = ["staticlib"]); viene trasformata in un file .lib in Windows e .a in Linux e macOS; può essere usata da programmi scritti in altri linguaggi
 - **cdylib**: libreria dinamica conforme alle specifiche C (crate-type = ["cdylib"]); viene trasformata in un file .dll in Windows, .so in Linux, .dylib in macOS; può essere usata da programma scritti in altri linguaggi

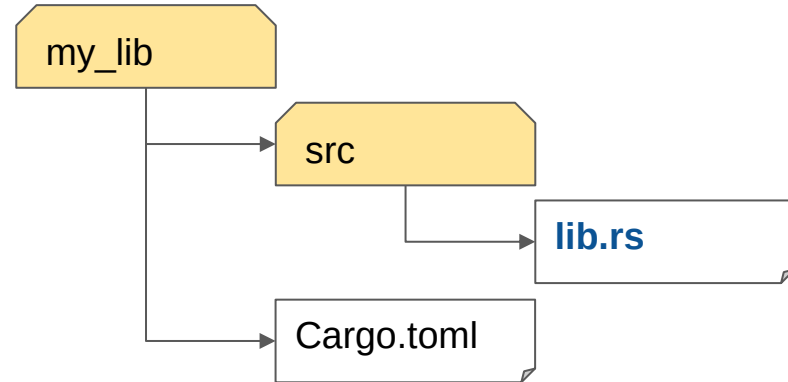
Libreria Statica Rust

- Creare una libreria Rust (tipo rlib) e usarla in un altro progetto Rust
- Creare un nuovo progetto per generare una libreria:
 - Da linea di comando: `cargo new my_lib --lib`
 - In Rust Rover



Scrittura della libreria

- Rust crea la struttura



- La scrittura della libreria richiede la definizione di una funzione pubblica.

```
pub fn add(left: u64, right: u64) -> u64 {  
    left + right  
}
```



Scrittura del Progetto

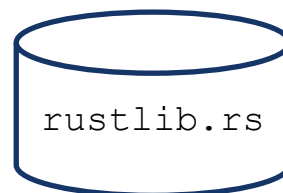
- Il file Cargo.toml deve dichiarare la dipendenza con una libreria esterna:

```
[dependencies]
```

```
my_lib = {path = "../my_lib"}
```

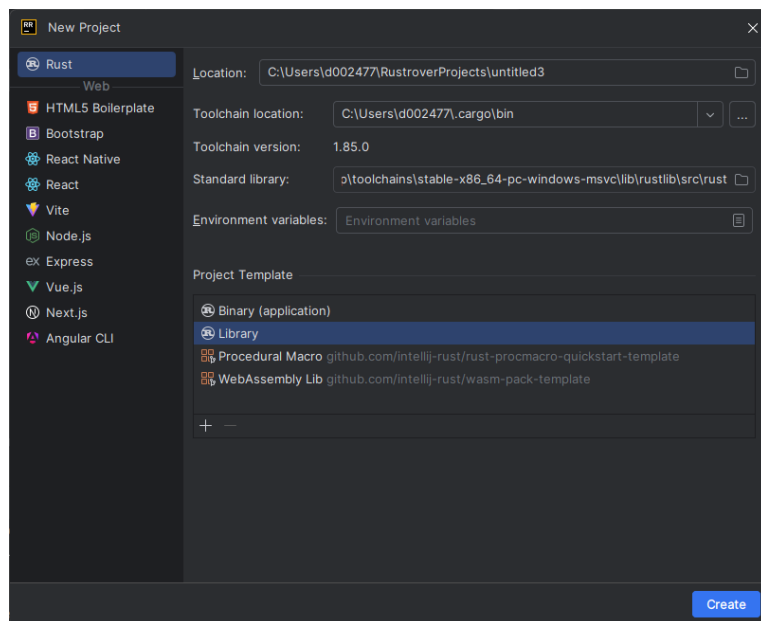
- main.rs deve includere la libreria esterna.

```
use my_lib;  
  
fn main() {  
    println!("Hello, world! {}", my_lib2::add(1, 2));  
}
```



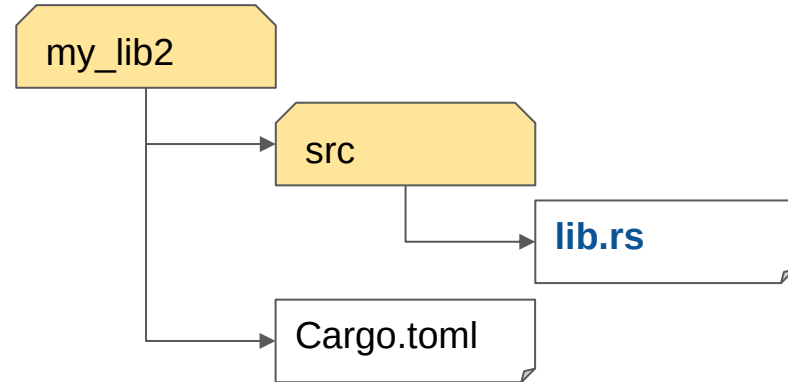
Libreria Dinamica Rust

- Creare una libreria Rust (tipo dylib) e usarla in un altro progetto Rust
- Creare un nuovo progetto per generare una libreria:
 - Da linea di comando: `cargo new my_lib --lib`
 - In Rust Rover



Scrittura della libreria

- Rust crea la struttura



- La scrittura della libreria richiede la definizione di una funzione pubblica.

```
pub fn somma(left: u64, right: u64) -> u64 {  
    left + right  
}
```



Scrittura della Libreria

- Il file Cargo.toml deve dichiarare la dipendenza con una libreria esterna:

```
[lib]
```

```
crate-type = ["dylib"]
```

Scrittura del Progetto

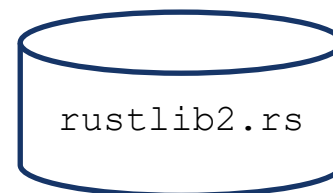
- Il file Cargo.toml deve dichiarare la dipendenza con una libreria esterna:

```
[dependencies]
```

```
my_lib2 = {path = "../my_lib2"}
```

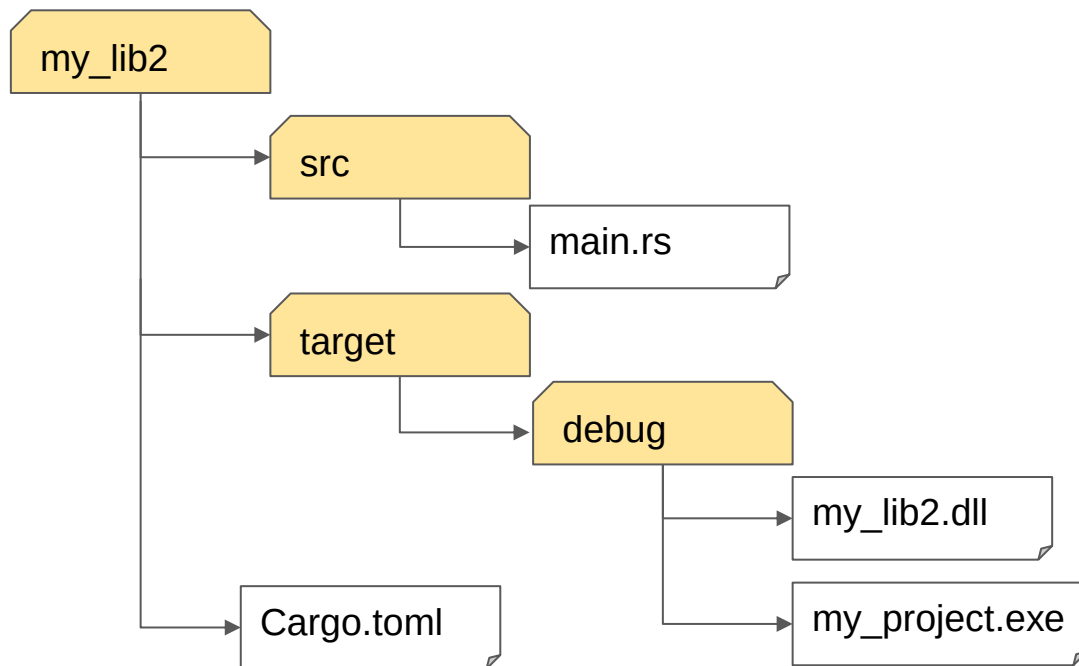
- main.rs deve includere la libreria esterna.

```
use my_lib2;  
  
fn main() {  
    println!("Hello, world! {}", my_lib2::somma(1, 2));  
}
```



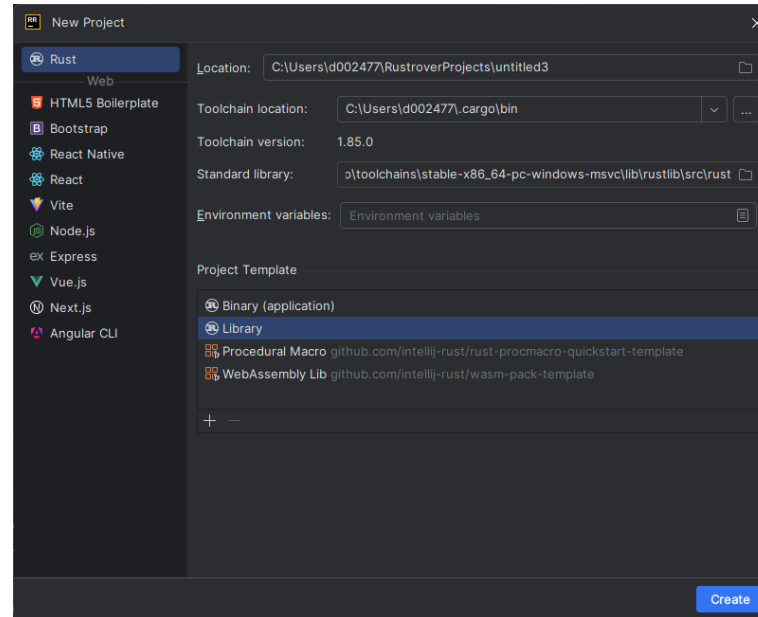
Generazione della libreria dinamica

- La compilazione della libreria o del progetto contenente il main genera la libreria con **lib**nomeliberia.**so** (Linux) o nome.**dll** (Windows).



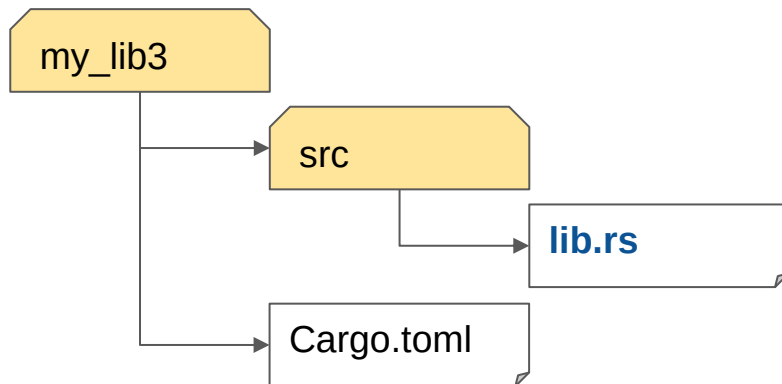
Libreria Statica Compatibile C

- Creare una libreria Rust (tipo staticlib) e usarla in un altro progetto Rust
- Creare un nuovo progetto per generare una libreria:
 - Da linea di comando: `cargo new my_lib --lib`
 - In Rust Rover



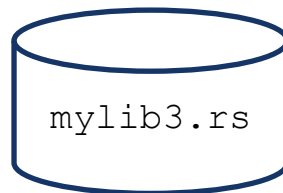
Scrittura della libreria

- Rust crea la struttura



- La scrittura della libreria richiede la definizione di una funzione pubblica esterna compatibile con il linguaggio C.

```
#[no_mangle]  
// impedisce a Rust di rinominare la funzione durante la compilazione  
pub extern "C" fn somma(left: u64, right: u64) -> u64 {  
    left + right  
}
```



Scrittura della Libreria

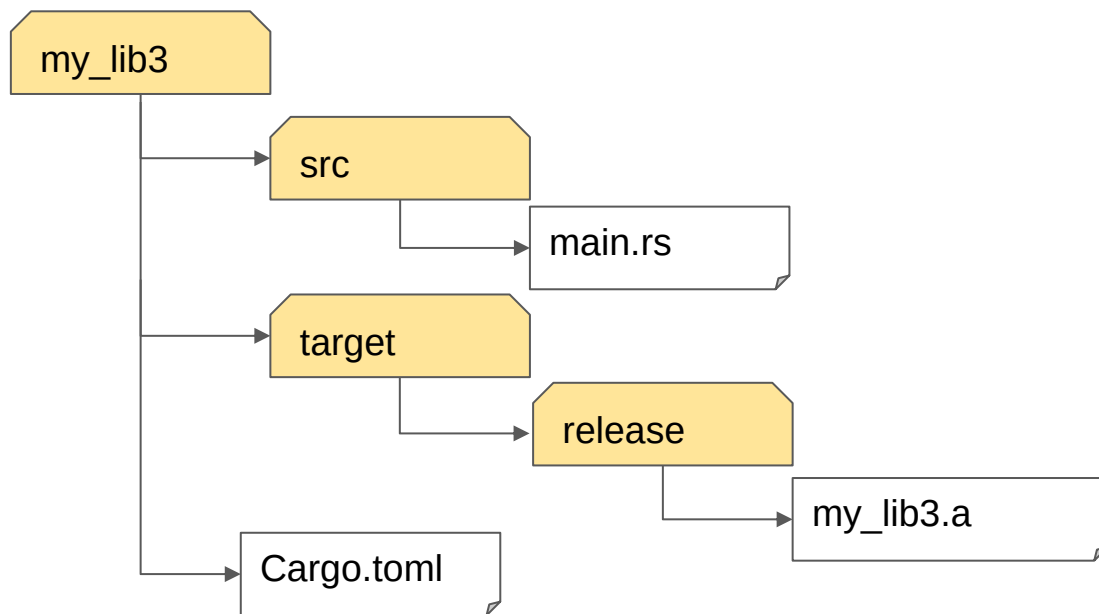
- Il file Cargo.toml deve dichiarare la dipendenza con una libreria esterna:

```
[lib]
```

```
crate-type = ["staticlib"]
```

Generazione della libreria

- Compila la libreria:
`cargo build --release`
- La compilazione della libreria o del progetto contenente il main genera la libreria con **lib**nomeliberia.**a** (Linux) o nome.**lib** (Windows).



Codice C

```
#include <stdio.h>

extern int add(int a, int b)

int main() {
    printf("3 + 4 = %d\n", add(3, 4));
    return 0;
}
```

- Compilazione del programma C linkando la libreria Rust

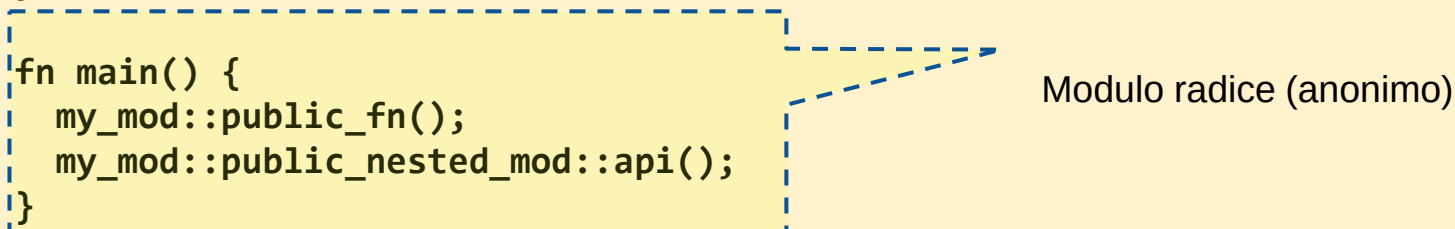
```
gcc main.c -L./target/release -lmy_lab3 -o main
```

Moduli e visibilità

- Il codice contenuto in un crate è costituito da un albero di moduli
 - Un modulo è un costrutto sintattico, introdotto dalla parola chiave `mod` e dal relativo nome che racchiude funzioni, tipi definiti dall'utente (`struct`, `enum`, `union`), tratti, implementazioni ed altri moduli
 - Tutto il codice che non è racchiuso in un blocco di tipo `mod nome_modulo { ... }`, fa parte del modulo radice
- I moduli formano una gerarchia ad albero, la cui radice è l'unità di compilazione corrente (rappresentata dalla parola chiave `crate`)
 - Di base, tutti gli elementi definiti all'interno di un modulo sono considerati privati, e sono accessibili solo al codice presente nel modulo stesso o nei suoi sotto-moduli
 - Se un elemento definito all'interno di un modulo "m1" è preceduto dalla parola chiave `pub`, diventa pubblico e può essere utilizzato dal codice presente in un modulo "m2" che non sia un suo discendente, a condizione che tutti gli antenati del modulo "m1" siano a loro volta accessibili al modulo "m2" (perché pubblici o perché il modulo "m2" è contenuto al loro interno)

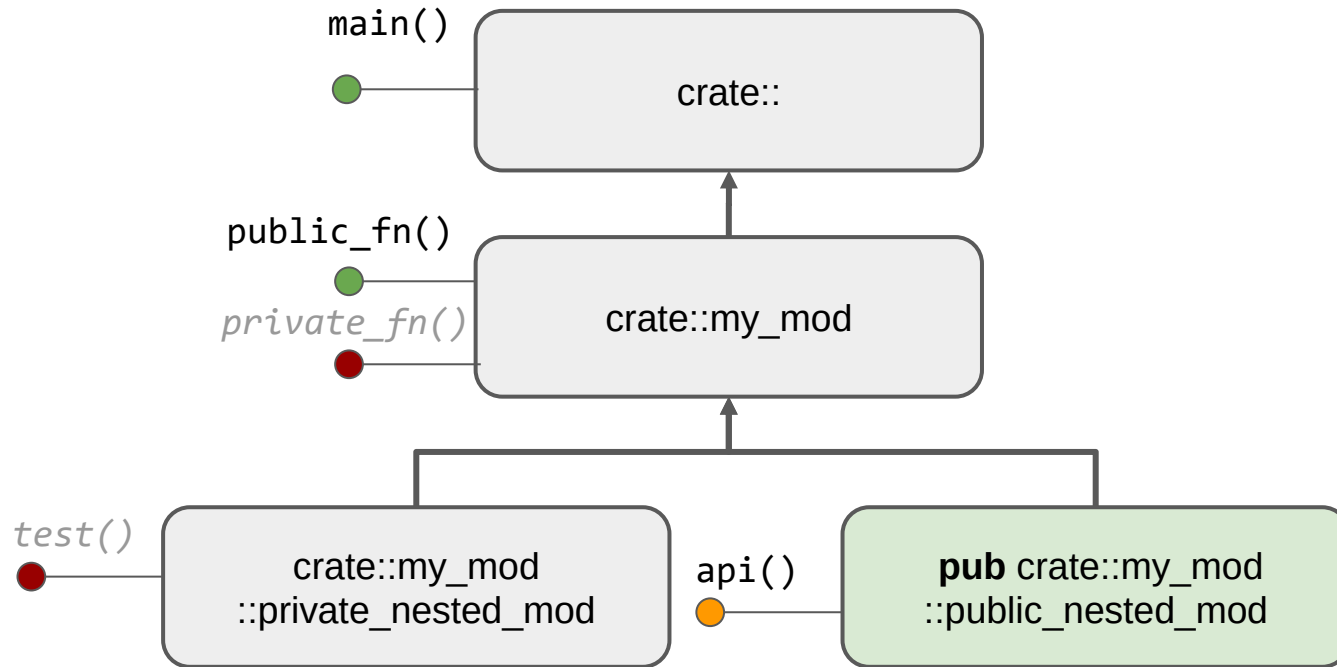
Moduli e visibilità

```
mod my_mod {  
  fn private_fn() {} // visibile solo nel modulo corrente  
  pub fn public_fn() {} // visibile nel contesto dell'unità di compilazione:  
                        // - codice presente in questo stesso file  
                        // - codice presente in un file che dichiara use my_mod;  
  mod private_nested_mod { // sotto-modulo privato  
    fn test() {} // può accedere a my_mod::private_fn()  
  }  
  pub mod public_nested_mod { //accessibile a chi ha accesso a my_mod  
    pub fn api() {}  
  }  
}  
  
fn main() {  
  my_mod::public_fn();  
  my_mod::public_nested_mod::api();  
}
```



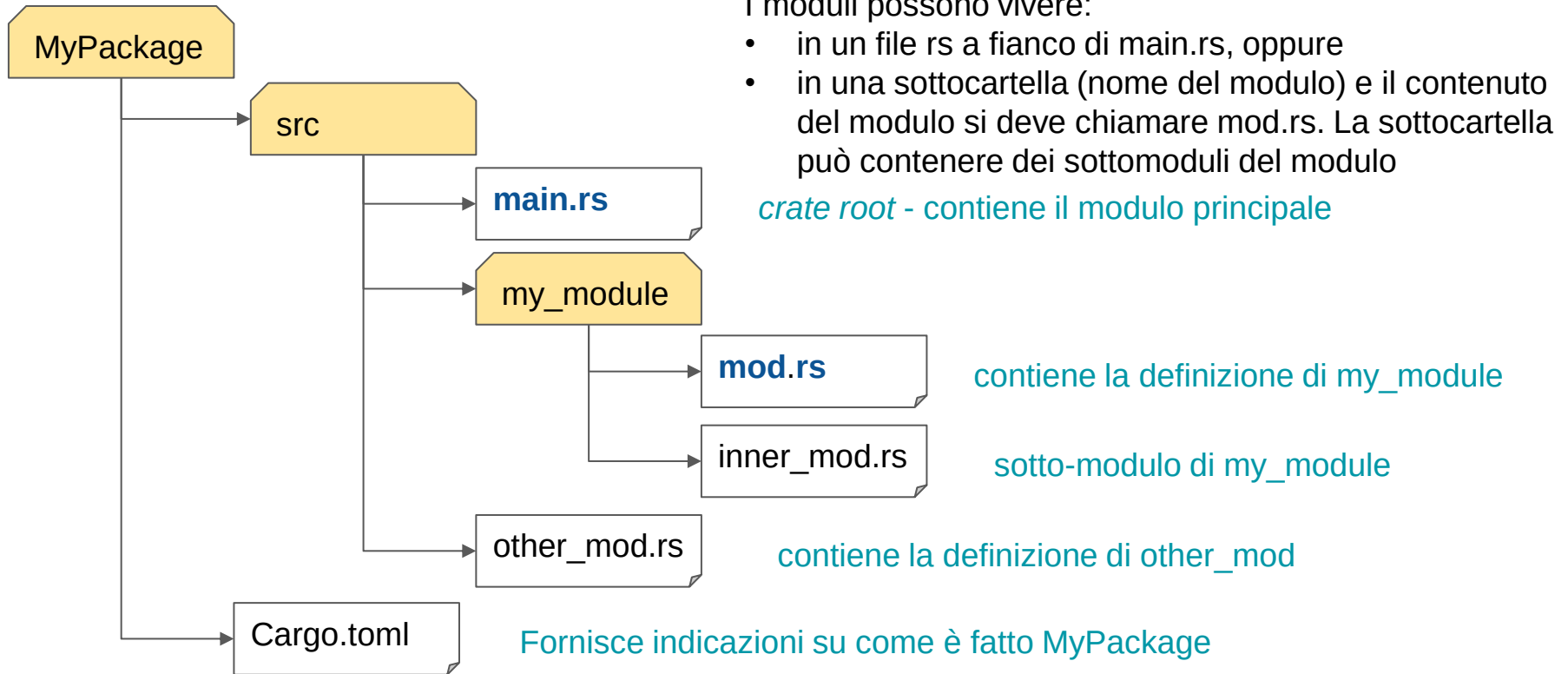
Modulo radice (anonimo)

Moduli e visibilità



Moduli e visibilità

- Per poter usare un simbolo (funzione, tipo, tratto, ...) definito in un modulo diverso da quello corrente occorre indicare al compilatore dove andare a reperirlo, facendo precedere il simbolo dal relativo cammino
 - Questo può essere assoluto o relativo
- Un cammino assoluto comincia con il nome del crate in cui si trova il simbolo seguito dalla sequenza gerarchica di moduli che occorre attraversare per giungere al simbolo
 - `let size = std::mem::size_of_val(&f);`
 - Si usa il separatore `::` per connettere i nomi dei (sotto-)moduli
 - Nel caso il simbolo appartenga al crate corrente, si può indicare come elemento iniziale la parola chiave `crate`
- Un cammino relativo comincia con le parole chiave `self`, `super` o direttamente con il nome del sottomodulo in cui il simbolo è definito
 - `let result = self::my_mod::public_nested_mod::api(some, arguments);`
- Per evitare di dover ripetere l'intero cammino per ogni occorrenza del simbolo, è possibile usare il costrutto `use cammino::del::modulo::*;`
 - Questo rende disponibile, nel file corrente, la definizione di tutti i simboli pubblici del modulo indicato
- Per poter accedere ad un simbolo occorre che tutti gli elementi del cammino siano accessibili a chi ne fa richiesta
 - Ovvero essere dichiarati come pubblici o contenere il modulo corrente



I moduli possono vivere:

- in un file rs a fianco di main.rs, oppure
- in una sottocartella (nome del modulo) e il contenuto del modulo si deve chiamare mod.rs. La sottocartella può contenere dei sottomoduli del modulo

crate root - contiene il modulo principale

contiene la definizione di **my_module**

sotto-modulo di **my_module**

contiene la definizione di **other_mod**

Fornisce indicazioni su come è fatto **MyPackage**

Moduli e file sorgente

- Il codice relativo ad un sotto-modulo può essere inserito:
 - Nello stesso file sorgente del modulo genitore, con la notazione `mod nome_modulo { ... }`
 - In un file sorgente a sé, presente nella stessa cartella, chiamato `nome_modulo.rs`: in questo caso occorre che nel modulo genitore sia presente la dichiarazione del modulo, nel formato `mod nome_modulo;`
 - In una sotto-cartella chiamata `nome_modulo`, nel file chiamato `mod.rs`: questo può contenere direttamente la definizione degli elementi che lo compongono o fare riferimento ad ulteriori file sorgente, posti nella stessa sotto-cartella, contenenti ciascuno un sotto-modulo
- Eventuali crate esterni, non facenti parte della gerarchia di moduli definiti dal package corrente, vengono elencati nel file `cargo.toml` che descrive la struttura del package stesso
 - Nella sezione `[dependencies]` sono presenti, uno per riga, i nomi dei crate da importare con l'indicazione della relativa versione

as

```
mod modulo_a {  
    pub fn funzione() {  
        println!("Funzione da modulo A");  
    }  
}  
  
mod modulo_b {  
    pub fn funzione() {  
        println!("Funzione da modulo B");  
    }  
}  
  
use modulo_a::funzione as funzione_a;  
use modulo_b::funzione as funzione_b;  
  
fn main() {  
    funzione_a();  
    funzione_b();  
}
```

*

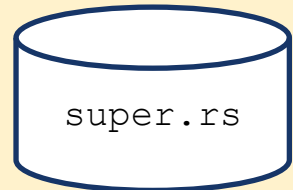
```
mod utilita {  
    pub fn funzione_a() {}  
    pub fn funzione_b() {}  
}  
  
use utilita::*;  
  
fn main() {  
    funzione_a();  
    funzione_b();  
}
```

```
mod genitore {  
    pub fn funzione_genitore() {  
        funzione_2();  
        println!("Sono nel modulo genitore.");  
    }  
    fn funzione_2() {  
        println!("Sono in una funzione del modulo genitore.");  
    }  
    pub mod figlio {
```

All'interno di un modulo, si può usare:

- self: per riferirsi al modulo corrente
- super: per riferirsi al modulo genitore.

```
        pub fn funzione_figlio() {  
            super::funzione_2();  
            println!("Sono nel modulo figlio.");  
            super::funzione_genitore(); // Chiamata a una funzione del modulo genitore  
            self::mia_funzione();       // Chiamata a una funzione nello stesso modulo  
        }  
        fn mia_funzione() {  
            println!("Funzione interna al modulo figlio.");  
        }  
    }  
}  
  
use genitore::figlio::funzione_figlio;  
fn main() {  
    funzione_figlio();  
}
```



Preludio

- Alcuni simboli particolarmente frequenti (come Vec, String, ...) non necessitano di essere importati in modo esplicito, pur appartenendo a crate distinti da quello corrente (sono infatti parte della libreria standard)
 - Questi sono elencati all'interno di un modulo chiamato `std::prelude` che viene importato automaticamente all'atto della compilazione di un file sorgente
 - In questo modo si evita di rendere verboso il codice sorgente
- Ogni versione di Rust viene con un proprio preludio
 - `std::prelude::v1`, `std::prelude::rust_2015`, `std::prelude::rust_2018`, `std::prelude::rust_2021`, `std::prelude::rust_2024`
 - Cargo provvede ad includere quello corretto in funzione del valore dell'attributo package/edition nel file `Cargo.toml`

Per saperne di più

- Rust Adventures: Rust projects management, understanding packages, Crates and modules
 - <https://levelup.gitconnected.com/rust-adventures-rust-projects-management-understanding-packages-crates-and-modules-b3bcde2eb1c>
- Rust Modules and Organizing Code
 - <https://medium.com/@TechSavvyScribe/rust-modules-and-organizing-code-92c4d6165a82>
- Module prelude
 - <https://doc.rust-lang.org/std/prelude/index.html>
- The edition guide
 - <https://doc.rust-lang.org/stable/edition-guide/>